CSC455 – Computer Vision

Name: Aoun Haider

ID: FA21-BSE-133

Submitted to: Dr. Zeeshan Gillani

Bloom Taxonomy Level: <*Applying*>

Question 1: [20 marks]

Create a 5x5 image matrix of your own choice with arbitrary pixels values within 0-100 range.

Apply Harris corner detection algorithm to compute the Harris corner response R for at least 3

pixels in the image, and classify each pixel into one of the following 3 categories:

- An edge pixel
- A corner pixel
- A flat region pixel

Note that your image must have at least 1 example of each of the above pixel categories. Show

the step-by-step calculation of R for each pixel.

Also note that each student should use a unique image matrix. A duplicate image matrix will be

considered as a case of plagiarism.

$R = +67600$ "Corner"

pixel (3,3) = 15

(1) Gradient calculation:

$I_x = (-1\times40) + (0\times85) + (1\times35) + (-2\times65) + (0\times15) + (2\times5) + (-1\times25) + (0\times30) + (1\times40)$

$I_x = -110$

$I_y = (-1\times40) + (-2\times85) + (-1\times35) + (0\times65) + (0\times15) + (0\times5) + (1\times25) + (2\times30) + (1\times40)$

$I_y = -120$

(2) Structure matrix

$A = I_x^2 = (-110)^2 = 12100$

$B = I_y^2 = (-120)^2 = 14400$

$C = I_x I_y = -110 \times -120 = 13200$

(3) Corner Response (R)

$R = (AB - C^2) - k(A+B)^2$

$R = (12100 \times 14400 - 13200^2) - 0.04 \times (12100 + 14400)^2$

$R = 0$ Flat edge

pixel 1 = edge, pixel 2 = corner

pixel 3 = Flat

$C = I_x \times I_y = 50 \times 60 = 3000$

(3) Compute corner response (R):

$R = \det(M) - R(\text{trace}(M))^2$

$= (A \cdot B - C^2) - 0.04 (2500 + 3600)^2$

$R = -1488400$

$R < 0$, so its an edge

Pixel (2,2) = 40

(1) Gradient Calculation:

$I_x = (-1 \times 20) + (0 \times 50) + (1 \times 90) + (-2 \times 60) + (0 \times 40) + (2 \times 75) + (-1 \times 95) + (0 \times 65) + (1 \times 15)$

$\boxed{I_x = -20}$

$I_y = (-1 \times 20) + (-2 \times 50) + (-1 \times 90) + (0 \times 60) + (0 \times 40) + (0 \times 85) + (1 \times 95) + (2 \times 65) + (1 \times 15)$

$\boxed{I_y = -30}$

(2) Structure matrix

$A = I_x^2 = (-20)^2 = 400$

$B = I_y^2 = (-30)^2 = 900$

$C = I_x I_y = (-20)(-30) = 600$

(3) Harris corner response (R):

$R = (A \cdot B - C^2) - K(A + B)^2$

$R = (400 \times 900 - 600^2) - 0.04 (400 + 900)^2$

Question 1:-

$$\text{Image} = \begin{bmatrix} 45 & 20 & 70 & 60 & 10 \\ 30 & 80 & 50 & 90 & 25 \\ 55 & 60 & 40 & 85 & 35 \\ 75 & 95 & 65 & 15 & 5 \\ 20 & 10 & 25 & 30 & 40 \end{bmatrix}$$

Gradient using sobel operator

$$I_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad I_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

pixel 1: (1,1) = 80

(1) Convolve soble kernel

$$I_x = (-1\times45) + (0\times20) + (1\times70) + (-2\times30) + (0\times80) + (2\times50) + (-1\times55) + (0\times60) + (1\times40)$$

$$I_x = 50$$

$$I_y = (-1\times45) + (-2\times20) + (-1\times70) + (0\times30) + (0\times20) + (0\times50) + (1\times55) + (2\times60) + (1\times40)$$

$$I_y' = 60$$

(2) Compute Structure matrix

$$A = I_x^2 = 50^2 = 2500$$

$$B = I_y^2 = 60^2 = 3600$$

Question 2: **[5 marks]**

```
import numpy as np


# Define the 5x5 image matrix
image = np.array([
    [20, 25, 20, 25, 20],
    [25, 80, 75, 80, 25],
    [20, 75, 50, 75, 20],
    [25, 80, 75, 80, 25],
    [20, 25, 20, 25, 20]
])


# Sobel kernels for gradient calculation
sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]])
sobel_y = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]])


def custom_convolve(image, kernel):
    """Custom function to perform convolution without using built-in functions."""
    image_height, image_width = image.shape
    kernel_height, kernel_width = kernel.shape
```

```
    output = np.zeros((image_height, image_width))

    # Compute padding sizes
    pad_height = kernel_height // 2
    pad_width = kernel_width // 2

    # Pad the image to handle border pixels
    padded_image = np.pad(image, ((pad_height, pad_height), (pad_width, pad_width)), mode='constant', constant_values=0)
    # Perform convolution using nested loops
    for i in range(image_height):
        for j in range(image_width):
            # Extract the region of interest from the padded image
            region = padded_image[i:i + kernel_height, j:j + kernel_width]
            # Compute the element-wise multiplication and sum the results
            output[i, j] = np.sum(region * kernel)

    return output

# Compute gradients using the custom convolution function
I_x = custom_convolve(image, sobel_x)
```

```
I_y = custom_convolve(image, sobel_y)

# Compute products of derivatives
I_x2 = I_x ** 2
I_y2 = I_y ** 2
I_xy = I_x * I_y

# Harris response calculation without Gaussian smoothing
k = 0.04  # Harris corner constant

# Direct calculation of det(M) and trace(M)
det_M = I_x2 * I_y2 - I_xy ** 2
trace_M = I_x2 + I_y2

# Harris response using the formula: R = det(M) - k * (trace(M))^2
harris_response = det_M - k * (trace_M ** 2)

print(harris_response)
# Initialize an empty matrix to store classifications
classification_matrix = np.empty(harris_response.shape,
dtype=object)
```

```
# Classify each pixel and store the result in the classification matrix
for i in range(harris_response.shape[0]):
    for j in range(harris_response.shape[1]):
        R = harris_response[i, j]
        if R > 0:
            classification_matrix[i, j] = "Corner"
        elif R < 0:
            classification_matrix[i, j] = "Edge"
        else:
            classification_matrix[i, j] = "Flat"

# Output the classification matrix
print("Classification Matrix:")
print(classification_matrix)
```